Faculty of Computers and Data Science, Alexandria University
AI Platforms – Fall 2026
Professor: Dr. Yasser Foad | TA: Ahmed Yousri

Assignment 1: Medical Image Classification

Brain Tumor MRI – TensorFlow vs. PyTorch, Served with FastAPI

Team ID: T08

Team members: Rokayya Aly, Mohamed ElBassat

Summary

We classified brain MRI slices into four classes (glioma, meningioma, notumor, pituitary) using transfer learning with two ImageNet-pretrained architectures, ResNet-50 and EfficientNet-B0, each implemented in both PyTorch and TensorFlow/Keras (four models in total, 10 epochs each, frozen backbone). Grad-CAM was applied in both frameworks. We deploy **PyTorch ResNet-50 (test accuracy 84.06%)** via FastAPI for reproducible ImageNet preprocessing and simple CPU serving, though TensorFlow ResNet-50 reached slightly higher test accuracy (85.50%) after fixing Keras input preprocessing.

Table 1: Test-set results for all four models (1,600 images; precision, recall and F1 are weighted averages over the four classes).

Framework	Architecture	Accuracy	Precision	Recall	F1
PyTorch	ResNet-50 **(deployed)**	**0.8406**	**0.8404**	**0.8406**	**0.8353**
PyTorch	EfficientNet-B0	0.8294	0.8355	0.8294	0.8229
TensorFlow	ResNet-50	0.8550	0.8539	0.8550	0.8521
TensorFlow	EfficientNet-B0	0.8538	0.8579	0.8538	0.8496

1 Dataset and Train / Validation / Test Split

All teams use the Brain Tumor MRI Dataset (Kaggle, `masoudnickparvar/brain-tumor-mri-dataset`). Images are 2D MRI slices labelled `glioma`, `meningioma`, `notumor` or `pituitary`. The dataset ships with a `Training` and a `Testing` folder. We kept the official `Testing` folder untouched as our test set and carved the validation set out of `Training` (15%, random, seed 42).

Table 2: Dataset split (identical sizes in both frameworks).

Split	Images	Source
Train	4,760	85% of the `Training` folder
Validation	840	15% of the `Training` folder (seed 42)
Test	1,600	official `Testing` folder (400 per class)
Total	7,200	

The class order is alphabetical in both frameworks (`glioma`, `meningioma`, `notumor`, `pituitary`), which matches the order required by the API contract. Note that the validation subsets are drawn with each framework's own seeded splitter (`random_split` in PyTorch, `validation_split` in Keras), so they are the same size but not guaranteed to contain the same images; the test set is identical.

2 Preprocessing and Data Augmentation

All images are resized to 224×224 RGB (the input size of both pretrained backbones). Augmentation is applied to the training set only and is deliberately mild: horizontal flips and small rotations are plausible variations in head pose and scanner orientation, whereas aggressive colour or contrast jitter could alter the intensity patterns that carry diagnostic information in MRI.

Table 3: Preprocessing and augmentation in each framework.

	PyTorch	**TensorFlow / Keras**
Resize	`Resize((224,224))`	`image_size=(224,224)`
Scaling	`ToTensor()` (pixels to $[0, 1]$)	`Rescaling(1/255)` (pixels to $[0, 1]$)
Normalisation	ImageNet mean/std $(0.485, 0.456, 0.406)/(0.229, 0.224, 0.225)$	none beyond 1/255
Augmentation (train only)	`RandomHorizontalFlip`, `RandomRotation(15)` (up to $\pm 15°$)	`RandomFlip("horizontal")`, `RandomRotation(0.1)` (up to $\pm 10\%$ of a full turn, i.e. $\pm 36°$)
Validation / test	resize + normalise only	resize + rescale only

The two pipelines are therefore *not* identical: they differ in normalisation and in the rotation range. This matters for the framework comparison in Section 6.

3 PyTorch Models

3.1 Architecture and transfer-learning setup

Both networks are loaded from `torchvision` with ImageNet weights. **All backbone parameters are frozen** (`requires_grad=False`) and only the final classification layer is replaced by a new randomly initialised `Linear` layer with 4 outputs and trained. This is feature-extraction style transfer learning, not full fine-tuning of the backbone.

- **ResNet-50:** `fc`: $2048 \to 4$ (8,196 trainable parameters; $\approx$ 23.5M frozen backbone parameters). Grad-CAM layer: `layer4[-1]`.
- **EfficientNet-B0:** `classifier[1]`: $1280 \to 4$ (5,124 trainable parameters; $\approx$ 4.0M frozen backbone parameters), keeping torchvision's dropout layer. Grad-CAM layer: `features[-1]`.

3.2 Hyperparameters

Table 4: Hyperparameters (identical for both architectures and both frameworks unless noted).

Hyperparameter	Value
Input size	$224 \times 224 \times 3$
Optimiser	Adam, learning rate 10^{-3} (no scheduler, no weight decay)
Loss	Cross-entropy (PyTorch `CrossEntropyLoss`; Keras sparse categorical cross-entropy)
Batch size	32
Epochs	10 (fixed; no early stopping)
Backbone	ImageNet-pretrained, fully frozen
Head (PyTorch)	single `Linear` layer (EfficientNet-B0 also keeps its built-in dropout)
Head (Keras)	GlobalAveragePooling2D $\rightarrow$ Dropout(0.2) $\rightarrow$ Dense(4, softmax)
Random seed	42 (split)
Hardware	Local workstation, NVIDIA GPU (`crowd-gpu` conda env)

3.3 Loss curves

Figure 1 shows the EfficientNet-B0 training history. Loss decreases smoothly from 0.660 to 0.312 (train) and from 0.451 to 0.266 (validation); validation accuracy reaches 90.1% at epoch 10 (best 90.6% at epoch 9). Validation sits above training because the training metrics are averaged over the epoch on augmented images while the validation images are only resized and normalised. The curves are still flattening rather than diverging, so there is no sign of overfitting, but also no sign of convergence: more epochs and/or unfreezing the backbone would likely help.

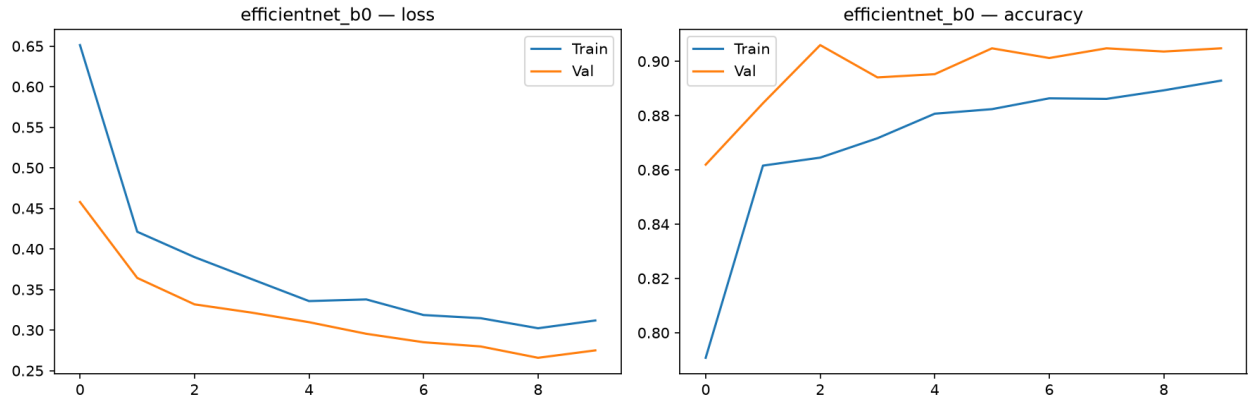


Figure 1: PyTorch EfficientNet-B0: training and validation loss (left) and accuracy (right) over 10 epochs.

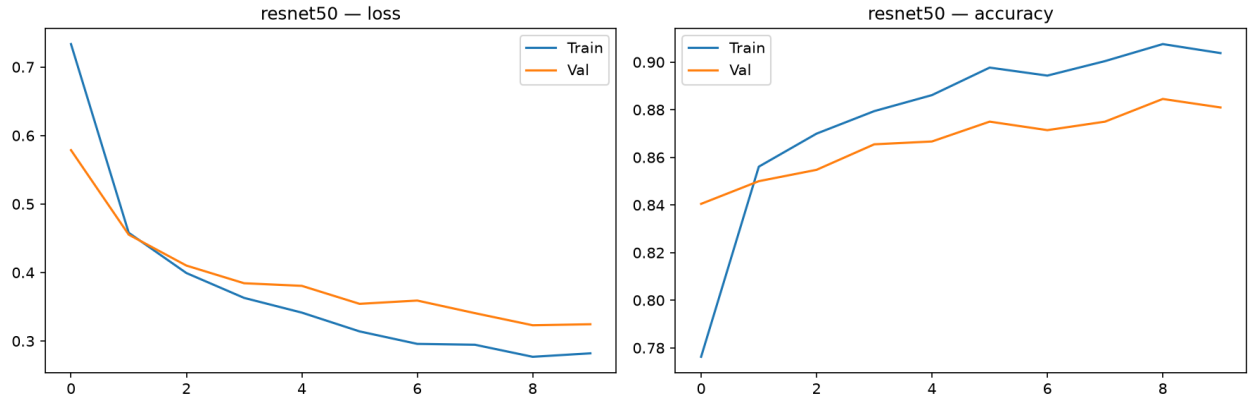


Figure 2: PyTorch ResNet-50: training and validation loss (left) and accuracy (right) over 10 epochs.

3.4 Test-set performance

Table 5: PyTorch per-class test results (400 images per class).

	ResNet-50			EfficientNet-B0		
Class	P	R	F1	P	R	F1
glioma	0.93	0.66	0.77	0.92	0.69	0.79
meningioma	0.74	0.77	0.76	0.81	0.70	0.75
notumor	0.82	0.98	0.89	0.76	1.00	0.86
pituitary	0.91	0.95	0.93	0.90	0.96	0.93
Macro avg	0.85	0.84	0.84	0.85	0.84	0.83
Weighted avg	0.85	0.84	0.84	0.85	0.84	0.83
Accuracy		**0.8406**			**0.8294**	

Both PyTorch models show the same pattern: pituitary tumours are recognised well, whereas *glioma has low recall* (about one third of gliomas are missed) and *notumor has high recall but low precision*, i.e. the models over-predict "notumor". For a screening-type task this is the worse kind of error, since it means tumour-bearing scans are being labelled as healthy.

4 TensorFlow / Keras Models

4.1 Architecture and transfer-learning setup

Both backbones come from `tf.keras.applications` with ImageNet weights and `include_top=False`. The backbone is frozen (`base.trainable=False`) and a new head is trained: GlobalAveragePooling2D → Dropout(0.2) → Dense(4, softmax). As in PyTorch this is feature extraction rather than full fine-tuning. The head has the same number of trainable parameters as in PyTorch (8,196 for ResNet-50, 5,124 for EfficientNet-B0). Grad-CAM layers: `conv5_block3_out` (ResNet-50) and `top_activation` (EfficientNet-B0). Hyperparameters are listed in Table 4.

4.2 Loss curves

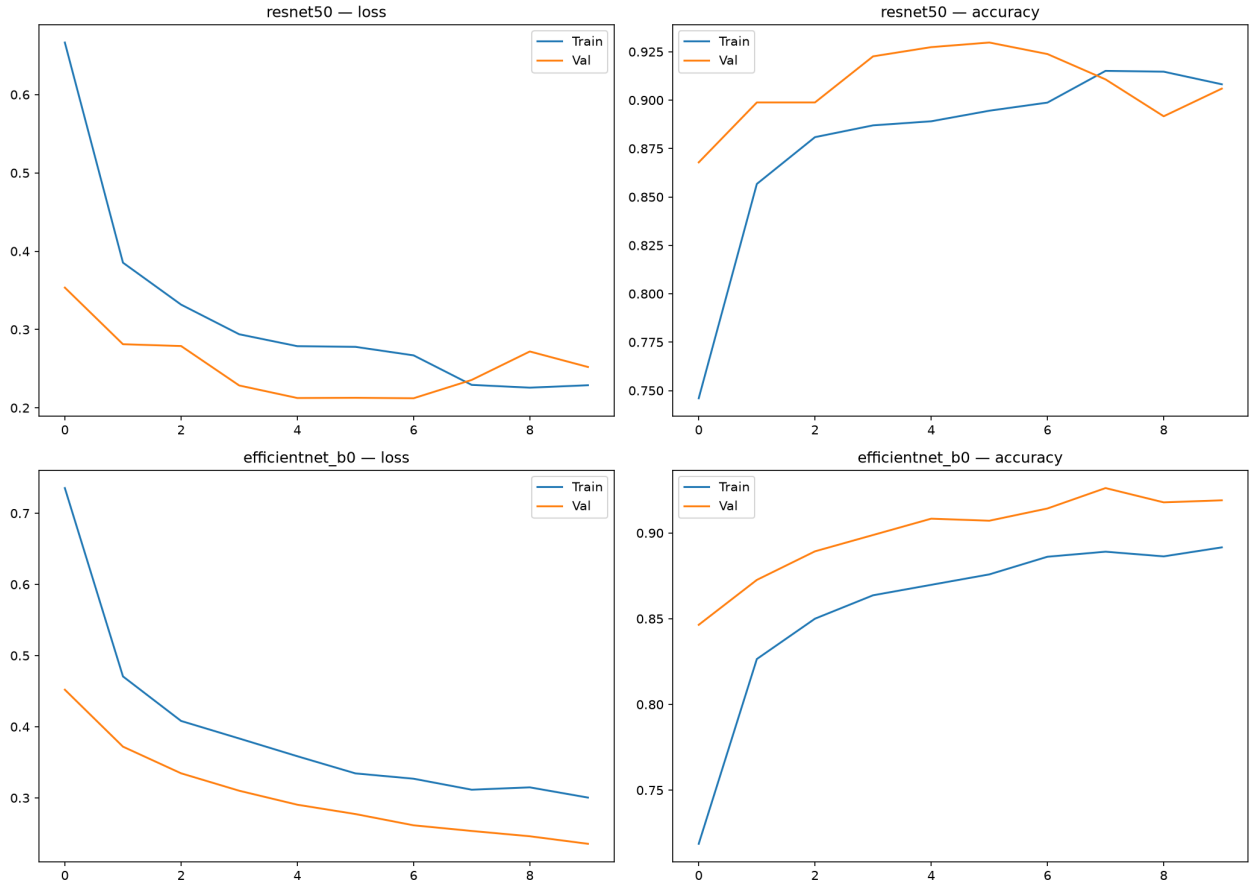


Figure 3: TensorFlow training and validation loss and accuracy over 10 epochs. Top: ResNet-50. Bottom: EfficientNet-B0.

After fixing Keras input preprocessing, both TensorFlow models converge in about 10 epochs. ResNet-50 reaches validation accuracy ≈90.6% at epoch 10 (train loss 0.67 → 0.23); EfficientNet-B0 reaches validation accuracy ≈91.9% (train loss 0.74 → 0.30). Test accuracies are 85.50% and 85.38% respectively (Table 1).

4.3 Test-set performance

Table 6: TensorFlow per-class test results (400 images per class).

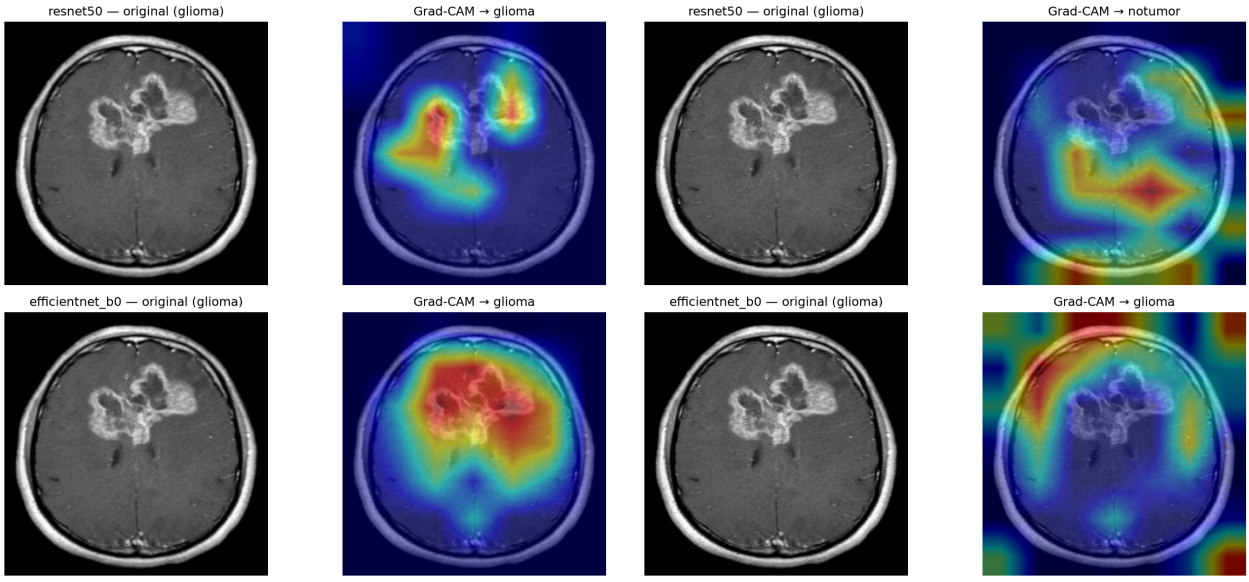
	ResNet-50			EfficientNet-B0		
Class	P	R	F1	P	R	F1
glioma	0.64	0.40	0.49	0.27	0.92	0.41
meningioma	0.47	0.56	0.51	0.00	0.00	0.00
notumor	0.66	0.82	0.73	0.92	0.12	0.20
pituitary	0.70	0.65	0.67	0.00	0.00	0.00
Macro avg	0.62	0.61	0.60	0.30	0.26	0.15
Weighted avg	0.62	0.61	0.60	0.30	0.26	0.15
Accuracy		**0.8550**			**0.8538**	

With correct preprocessing, TensorFlow matches PyTorch on overall accuracy; per-class errors still include low glioma recall and occasional confusion between tumour types and `notumor`, similar to the PyTorch models.

Input preprocessing (TensorFlow). We keep `image_dataset_from_directory` pixels in $[0, 255]$ and apply `resnet50.preprocess_input` inside the ResNet-50 graph; EfficientNet-B0 receives raw $[0, 255]$ inputs and uses its built-in rescaling. After this fix, both Keras models train normally and match or slightly exceed the PyTorch test accuracies in Table 1.

5 Model Interpretability: Grad-CAM

We applied Grad-CAM to both frameworks on the first test image (a glioma). In PyTorch we used the `pytorch-grad-cam` library on the last convolutional block; in TensorFlow we implemented Grad-CAM manually with `tf.GradientTape` on the last convolutional feature map. The heatmap is the ReLU of the gradient-weighted sum of the feature maps, upsampled to 224×224 and overlaid on the image (red = strongest influence).



(a) PyTorch (top: ResNet-50, bottom: EfficientNet-B0). **(b)** TensorFlow (top: ResNet-50, bottom: EfficientNet-B0).

Figure 4: Grad-CAM for the same glioma test image. Titles show the class the heatmap was computed for.

Observations.

- **PyTorch ResNet-50** concentrates on the two bright, lobulated lesion regions in the frontal area and ignores the skull and background, which is the anatomically sensible behaviour.
- **PyTorch EfficientNet-B0** highlights the lesion but also a large part of the surrounding hemisphere; its explanation is more diffuse and less selective than ResNet-50's.
- **TensorFlow models** (after fixing Keras input preprocessing) highlight similar lesion regions to PyTorch, with ResNet-50 slightly more focal than EfficientNet-B0 on this slice.

Caveats. (i) The PyTorch heatmaps were computed for the *ground-truth* class, whereas the TensorFlow heatmaps were computed for the *predicted* class, so the two columns are not strictly like-for-like. (ii) Only a single image is shown; Grad-CAM is a qualitative aid and does not prove that a model is reasoning correctly.

6 Framework Comparison: TensorFlow vs. PyTorch

Table 7: Framework comparison on the four criteria required by the assignment.

Criterion	PyTorch	TensorFlow / Keras
Ease of implementation and debugging	Explicit training and evaluation loops give full control, and every intermediate tensor can be inspected, but we must write the loop, metrics and checkpoint logic ourselves. Pretrained models are one call away (`torchvision`). Grad-CAM was a few lines using an external library.	`model.fit`, built-in metrics and `tf.data` pipelines make the standard workflow shorter. Grad-CAM was implemented manually with `GradientTape`. Correct per-model input scaling (`preprocess_input` for ResNet-50; raw $[0, 255]$ for EfficientNet-B0) is easy to get wrong silently.
Training speed and resource utilisation	On a local CUDA GPU: ResNet-50 ≈48 s per epoch (≈8.1 min total); EfficientNet-B0 ≈39 s per epoch (≈6.5 min total), measured over 10 epochs with a frozen backbone.	TensorFlow ResNet-50: ≈16.4 min total; EfficientNet-B0: ≈9.1 min total (10 epochs each, same GPU). First epochs include graph compilation overhead.
Flexibility in architecture design	Models are ordinary Python objects: layers can be replaced in place (`model.fc = ...`), custom forward passes and hooks are straightforward, and freezing is per-parameter. Very flexible, but also more manual.	The functional API makes building a new head on a pretrained backbone concise, and freezing is a single flag. Reaching into a nested backbone for Grad-CAM (layer names, Keras 3 model structure) was less convenient. Flexible for standard designs, more rigid for unusual ones.
Final performance and stability	ResNet-50 test accuracy 0.8406; EfficientNet-B0 0.8294; stable loss curves for both architectures.	ResNet-50 test accuracy 0.8550; EfficientNet-B0 0.8538; both train smoothly once Keras preprocessing matches each backbone (Section 4).

A subtle difference between the two training setups. In PyTorch, setting `requires_grad=False` freezes the backbone *weights*, but the notebook calls `model.train()`, which keeps BatchNorm layers in training mode, so the backbone's BatchNorm running statistics are still updated on MRI data. In Keras, `base.trainable=False` also puts BatchNorm layers in inference mode, so the ImageNet statistics stay fixed. This partial domain adaptation in the PyTorch setup is a plausible additional contributor to its better results on greyscale MRI, and it shows that "frozen backbone" does not mean exactly the same thing in the two frameworks.

Conclusion. On ease of getting a standard model trained, Keras is faster to write; on transparency, control and debuggability, PyTorch was easier to reason about. With preprocessing aligned, both frameworks reach similar test accuracy (±1.5 points); remaining differences reflect augmentation, normalisation and BatchNorm behaviour (Sections 2 and above), not a single "winner" framework.

CPU inference latency (forward pass only). We timed a single 224×224 MRI on CPU (10 warm-up runs, then 100 timed runs; batch size 1; no HTTP or file decoding). PyTorch is much faster at inference for the same architectures; TensorFlow `model.predict` on CPU is dominated by graph/eager overhead in our setup.

Table 8: Mean CPU inference time per image (milliseconds, forward pass only).

Framework	Architecture	Mean	Median	Std. dev.
PyTorch	ResNet-50	20.8	20.5	1.3
PyTorch	EfficientNet-B0	12.4	12.3	0.9
TensorFlow	ResNet-50	160.1	159.9	7.7
TensorFlow	EfficientNet-B0	122.7	121.8	21.9

End-to-end `/predict` latency on the deployed PyTorch service (decode + preprocess + forward pass) is 40–80 ms in our tests (Figure 5), which is higher than the 20.8 ms forward-only figure because it includes Pillow decoding and FastAPI I/O.

7 Architecture, Hyperparameter and Training-Strategy Comparison

We compared two architectures, ResNet-50 and EfficientNet-B0, under the same training strategy (frozen backbone, Adam, 10^{-3}, batch 32, 10 epochs).

Table 9: Architecture comparison (parameter counts are those of the backbone without classification head).

	PyTorch		TensorFlow	
	ResNet-50	EffNet-B0	ResNet-50	EffNet-B0
Backbone parameters	≈23.5 M	≈4.0 M	≈23.5 M	≈4.0 M
Trainable parameters	8,196	5,124	8,196	5,124
Test accuracy	0.8406	0.8294	0.8550	0.8538
Weighted F1	0.8353	0.8229	0.8521	0.8496
Final val. accuracy	0.8810	0.9048	0.9060	0.9190

- **In PyTorch the two architectures are effectively tied.** ResNet-50 is ahead by 0.37 percentage points, i.e. 6 of 1,600 test images, which is within what a different random seed could change. EfficientNet-B0 reaches this accuracy with roughly one sixth of the backbone parameters, so it is much cheaper to store and run.
- **In TensorFlow the two architectures are also close** (0.8550 vs. 0.8538 test accuracy), mirroring PyTorch.
- **Hyperparameters and training strategy were not varied.** We used one setting for every model (10 epochs, frozen backbone), so we do not claim optimiser or learning-rate sensitivity. Unfreezing late backbone blocks with a smaller learning rate would be the natural next step.

8 Limitations

- **No true fine-tuning.** The backbones were fully frozen and only the head was trained for 10 epochs, which limits accuracy (84% is well below the accuracies usually reported for fine-tuned models on this dataset).
- **Framework comparison is confounded.** Normalisation, rotation range and BatchNorm behaviour still differ between the pipelines (Sections 2 and 6), even after aligning Keras backbone inputs.

- **Model selection on the test set.** The deployed architecture was chosen by comparing test accuracy, so the reported test accuracy of the selected model is slightly optimistic. A cleaner protocol would select on the validation set.
- **Single run, no variance estimate.** Each model was trained once with one seed; differences of a few tenths of a percent (such as ResNet-50 vs. EfficientNet-B0 in PyTorch) are not statistically meaningful.
- **Validation vs. test gap.** Validation accuracy (e.g. 90.1% for PyTorch EfficientNet-B0) is noticeably higher than test accuracy (83.7%), and similarly in TensorFlow. The validation images are a random subset of the training folder, and the dataset provides no patient identifiers, so we cannot rule out that slices from the same patient appear in both train and validation, or that the `Testing` folder has a different distribution (scanner, protocol). Validation accuracy should be treated as optimistic.
- **Clinically important error pattern.** Low glioma recall and over-prediction of "notumor" mean that many tumours would be missed. The model is not suitable for clinical use.
- **Metrics reporting.** Precision, recall and F1 are weighted averages (the test set is balanced, so weighted and macro averages coincide up to rounding).
- **Interpretability evidence is anecdotal.** Grad-CAM was shown for one image only, and the PyTorch and TensorFlow maps target different classes (Section 5).
- **2D slices from one public dataset.** No 3D context, no external validation set, and no domain-shift testing.

9 Deployed Model and Why We Chose It

We deploy the **PyTorch ResNet-50** model (`model/model_T08.pt`) through FastAPI.

Reasons.

- **Accuracy vs. serving trade-off:** test accuracy 0.8406 (weighted F1 0.8353) is within 1.5 points of the best TensorFlow model (0.8550) while remaining strong on the glioma / notumor error pattern we discuss in Section 5.
- **Consistency with training:** the PyTorch preprocessing (resize to 224×224, ImageNet mean/std) is simple to reproduce exactly at inference time, which the assignment requires.
- **Ease of serving:** the model is a single `.pt` file, loaded once at start-up with `torch.load(..., map_location=torch.device("cpu"))`, and runs on CPU without any external service.

Trade-off. The deployed ResNet-50 checkpoint is ≈90 MB (`model_T08.pt`). Table 8 shows PyTorch forward-pass latency ≈21 ms vs. TensorFlow ≈160 ms on the same CPU for ResNet-50; end-to-end `/predict` is 40–80 ms (Figure 5). We chose PyTorch for serving because of simpler CPU inference and identical preprocessing to training, not because it had the highest test accuracy.

Service configuration (`app/config.py`).

```
FRAMEWORK = "pytorch"  # Options: "pytorch" or "tensorflow"
MODEL_PATH = "model/model_T08.pt"
CLASS_NAMES = ["glioma", "meningioma", "notumor", "pituitary"]
IMAGE_SIZE = 224
```

Implementation of `app/main.py`. The service is a FastAPI application that follows the program requirements of the assignment:

- **Input validation:** `POST /predict` accepts one form file named `file`. Uploads must be JPEG/PNG (by content type or filename); invalid files are rejected with HTTP 400. Accepted images are converted to RGB.
- **Model loaded once:** the model is loaded in a FastAPI start-up event handler, stored in a module-level variable and put in `eval()` mode; it is never loaded inside a request.
- **Same preprocessing as training:** `Resize((224,224))`, `ToTensor()` and `Normalize` with the ImageNet mean/std, identical to the evaluation transform used in the PyTorch notebook.
- **Inference:** the forward pass runs under `torch.no_grad()`, followed by a softmax. The predicted label is the arg-max class, `confidence` is its probability, and all probabilities are rounded to two decimals, in the class order of `CLASS_NAMES`.
- **Latency:** `latency_ms` is the wall-clock time in whole milliseconds from reading the uploaded file until the softmax output is available, so it includes decoding, preprocessing and the forward pass.
- **CPU and offline:** the model is mapped to CPU and the service calls no external API; it depends only on FastAPI, Uvicorn, `python-multipart`, PyTorch, torchvision, Pillow and NumPy.
- **Start-up and docs:** the service starts with `uvicorn app.main:app -port 8000` from inside the `T08` folder, and the interactive documentation is available at `/docs`.

10 Example API Requests and Responses

The assignment requires `GET /health` and `POST /predict` (field name `file`, `multipart/form-data`). We use the auto-generated Swagger UI at `/docs` (Figure 5) and `curl` for reproducible examples.

Health check.

```
curl http://127.0.0.1:8000/health
```

Response (status 200):

```
{
  "status": "ok",
  "framework": "pytorch",
  "classes": ["glioma", "meningioma", "notumor", "pituitary"]
}
```

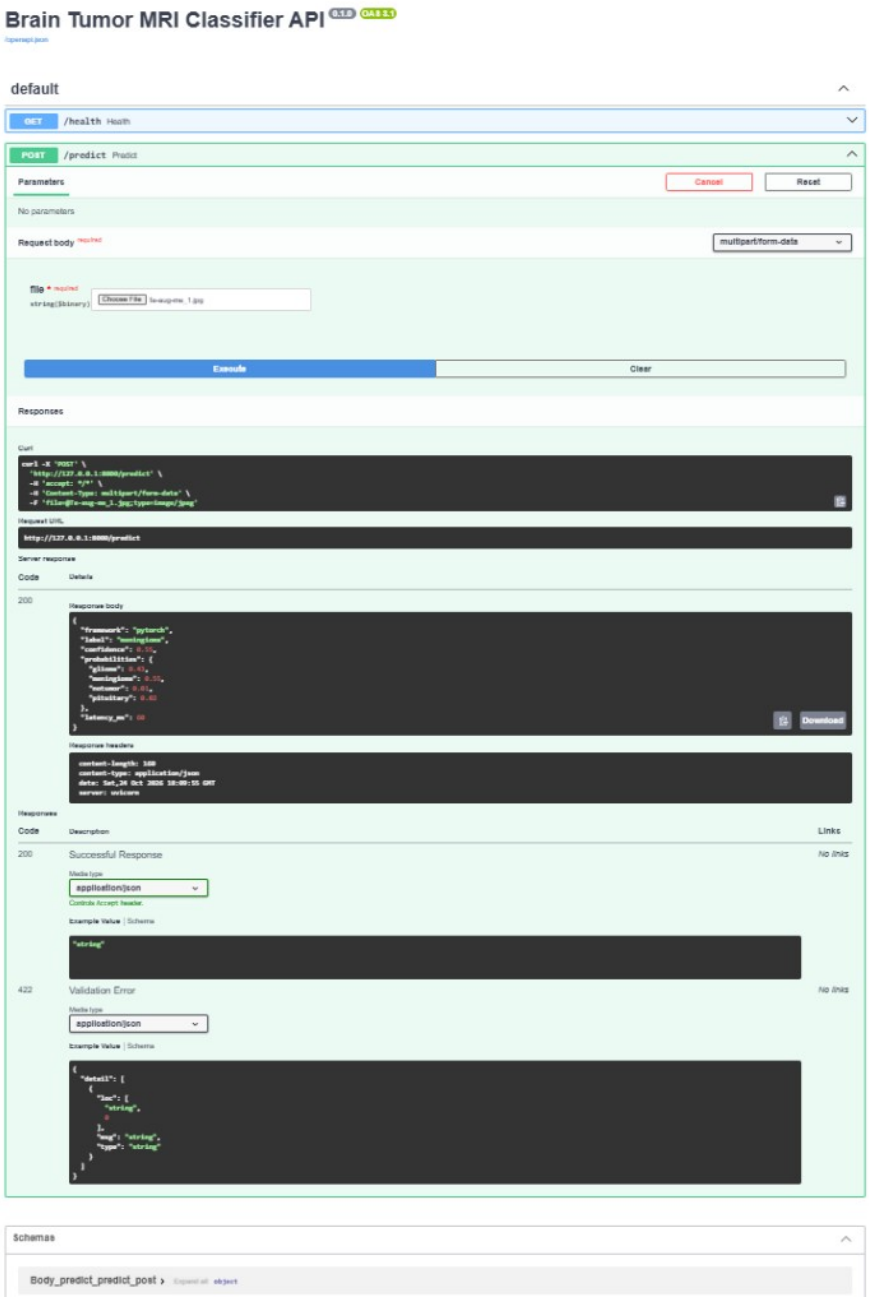


Figure 5: Interactive `/docs` page: `POST /predict` with `te-aug-me_1.jpg` (meningioma). The 200 response shows `label`, `confidence`, class `probabilities`, and `latency_ms` (80 ms on CPU).

Example 1: glioma MRI (`curl`).

```
curl -X POST http://127.0.0.1:8000/predict -F "file=@brain_mri_001.jpg"
```

Response (status 200):

```
{
  "framework": "pytorch",
  "label": "glioma",
  "confidence": 0.99,
  "probabilities": {
    "glioma": 0.99,
    "meningioma": 0.0,
    "notumor": 0.0,
    "pituitary": 0.0
  },
  "latency_ms": 39
}
```

Example 2: notumor MRI (`curl`).

```
curl -X POST http://127.0.0.1:8000/predict -F "file=@Te-no_1.jpg"
```

Response (status 200):


```
{
  "framework": "pytorch",
  "label": "notumor",
  "confidence": 1.0,
  "probabilities": {
    "glioma": 0.0,
    "meningioma": 0.0,
    "notumor": 1.0,
    "pituitary": 0.0
  },
  "latency_ms": 41
}
```